Informe de seguridad

PharmaBackend · Actividad autónoma · Sesión 10

Kevin Eduardo Del Carpio Alegría · Lenguaje de Programación II · Ciclo IV · 2026-2

C1. Resumen de verificación

Se ejecutaron 28 casos de autenticación, JWT, permisos, usuarios, CORS y administración de credenciales. Los rechazos 400, 401, 403, 404 y 409 previstos son resultados correctos: cada estado se compara con la regla del caso, no con la expectativa de obtener siempre 200.

Grupo	Ejecutados	Pasa	Falla
Autenticación (A)	5	5	0
Token (T)	6	6	0
Permisos (P)	5	5	0
Usuarios (U)	3	3	0
CORS (C)	1	1	0
Contraseña, roles y estado (B)	8	8	0
Total	28	28	0

La regla de username acepta mayúsculas y elimina espacios exteriores: AuthServiceImpl.login aplica trim y el repositorio busca sin distinguir mayúsculas. Los cuatro casos propios son A-05, T-06, P-05 y U-03. La cuenta autonoma_s10 (usuario 22, empleado 21, Maria Huaman Pruebas S10) permite probar la parte B conservando la cuenta previa de María.

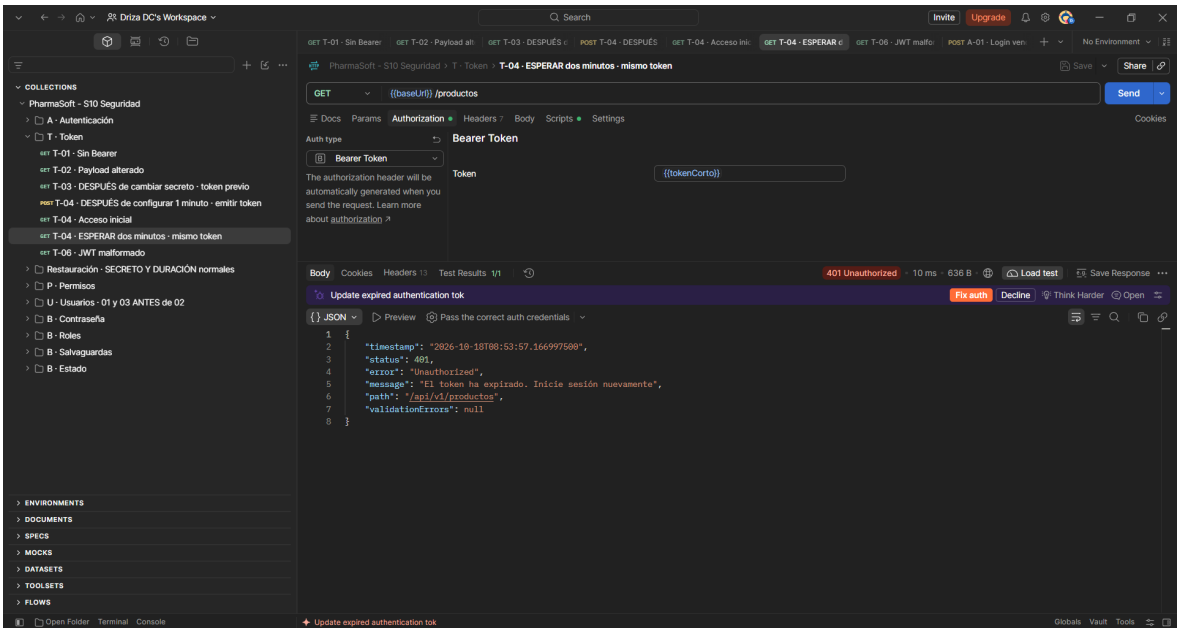


Figura 1. T-04: el token corto vencido recibe 401 con un mensaje específico de expiración.

La matriz consolida los resultados confirmados de ejecución. Las capturas documentan las respuestas HTTP; las comprobaciones complementarias de hashes y relaciones se identifican como pruebas automatizadas en H2. La regresión ejecutó 57 pruebas sin fallos; otras 10 pruebas Oracle optativas no se ejecutaron.

Implementación y comportamiento comprobado

Contraseña. AuthController.cambiarPassword obtiene el username de Authentication. AuthServiceImpl.cambiarPassword verifica la actual con PasswordEncoder.matches, rechaza la repetición y codifica la nueva con BCrypt. CambioPasswordRequestDTO exige entre 8 y 72 caracteres, letra y número; el servicio también controla 72 bytes UTF-8. B-01 a B-04 obtienen 409, 400 y 204 según la regla. El login antiguo pasa a 401, el nuevo a 200 y el JWT anterior conserva su validez. El log INFO del servicio registra el username sin incluir contraseñas.

Roles. UsuarioServiceImpl.asignarRoles carga y valida todos los ids y reemplaza el conjunto mediante setRoles dentro de una transacción. B-05 muestra 403 antes del cambio, 200 al asignar VENDEDOR y ALMACENERO y 200 al editar con el mismo token. Al reemplazar por solo ALMACENERO, clientes vuelve a 403. B-06 confirma 400 para conjuntos inválidos y 404 para ids inexistentes.

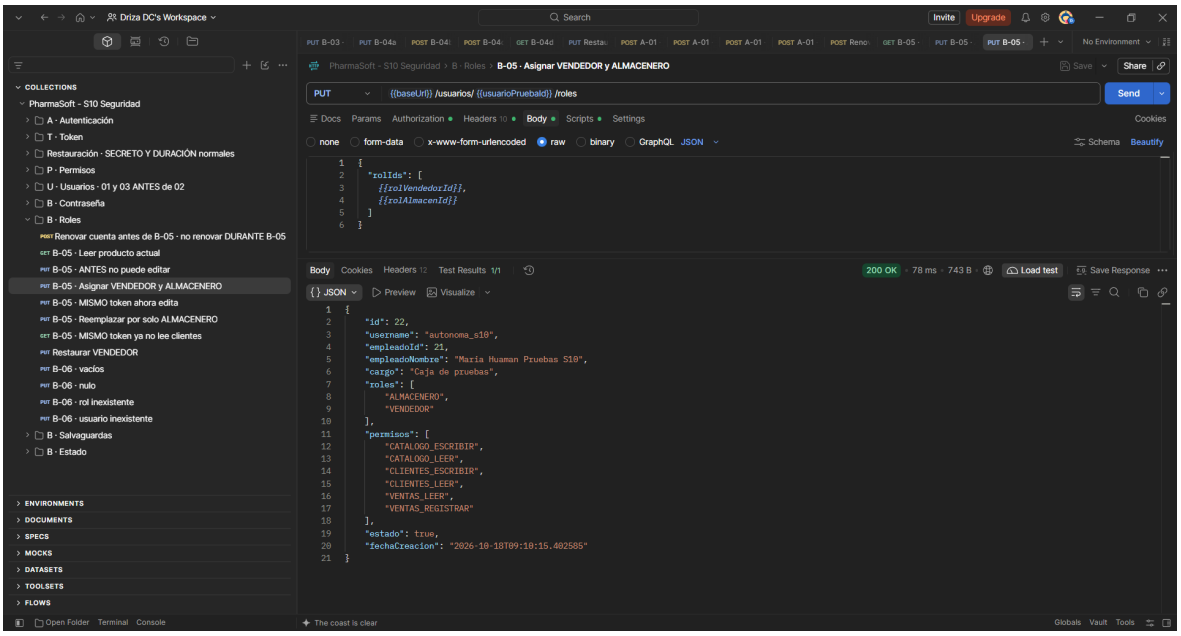


Figura 2. B-05: respuesta 200 con ambos roles y sus permisos efectivos.

Permisos actuales con el mismo JWT. JwtAuthenticationFilter.doFilterInternal valida el token, extrae username y llama a loadUserByUsername en cada petición. Construye Authentication con las autoridades actuales de la cuenta, no con una copia fija de los claims authorities. Por eso el cambio de roles aplica en la siguiente petición sin repetir el login.

Estado y salvaguardas. UsuarioServiceImpl.cambiarEstado conserva la cuenta y su historial. B-08 muestra 200 al desactivar, 401 con el token previo y 401 genérico en login; al reactivar, ambos accesos vuelven a 200 si el JWT sigue vigente. B-07 confirma 409 al autoquitar ADMIN o autodesactivarse; una cuenta sin USUARIOS_ADMINISTRAR recibe 403.

C2. Hallazgos

No se registraron casos funcionales fallidos en la ejecución consolidada: 28 pasan y 0 fallan. Por ello no corresponden fichas de defecto con severidad, causa probable y corrección. El intento inicial de CORS se repitió con la SPA iniciada y terminó con OPTIONS 200 y GET 200; fue una condición del entorno, no un defecto del backend.

C3. Análisis de seguridad

1. Mensaje uniforme de login. El usuario inexistente, la contraseña incorrecta y la cuenta deshabilitada reciben 401 con el mismo mensaje. Esto evita que un atacante use la respuesta para enumerar cuentas o conocer su estado. El manejo de errores de autenticación mantiene el contrato genérico, mientras JwtAuthenticationFilter distingue la deshabilitación al evaluar un JWT ya emitido; se separa así la información necesaria de una petición autenticada del diagnóstico público del login.

2. Token después del cambio de contraseña. Un JWT firmado sigue válido porque cambiar el hash no cambia su firma ni su fecha exp. AuthServiceImpl.cambiarPassword no mantiene sesiones ni revoca tokens, y JwtAuthenticationFilter comprueba firma, vigencia y cuenta habilitada. Una mejora sería añadir tokenVersion al usuario y al JWT, incrementarla al cambiar la contraseña y comparar versiones en cada petición; los tokens antiguos quedarían invalidados. No se implementó esa política porque B-04 verifica expresamente la continuidad del JWT anterior.

3. Secreto JWT expuesto. Quien conoce la clave de firma puede fabricar tokens válidos e intentar suplantar usuarios existentes; la recarga de permisos desde la base no evita la suplantación de un administrador. La respuesta inmediata consiste en rotar JWT_SECRET por una clave nueva, reiniciar o desplegar todas las instancias y exigir nuevos logins: T-03 demuestra el rechazo de tokens firmados con la clave anterior. Después corresponde retirar el secreto del repositorio y su historial, revisar accesos y mantener la clave fuera del código. JwtService y la configuración por entorno concentran esta gestión.

4. Autorización por método. @PreAuthorize en ReporteController expresa REPORTES_VER junto a las operaciones que protege y evita depender únicamente de la coincidencia de una URL en SecurityConfig. Facilita reconocer la regla al leer el controlador y probarla por operación. Como desventaja, distribuye las reglas entre clases, exige activar la seguridad de métodos y requiere cuidar la consistencia al agregar controladores. Las pruebas P-03 y P-04 contrastan el rechazo de vendedor/almacén con el acceso del administrador.

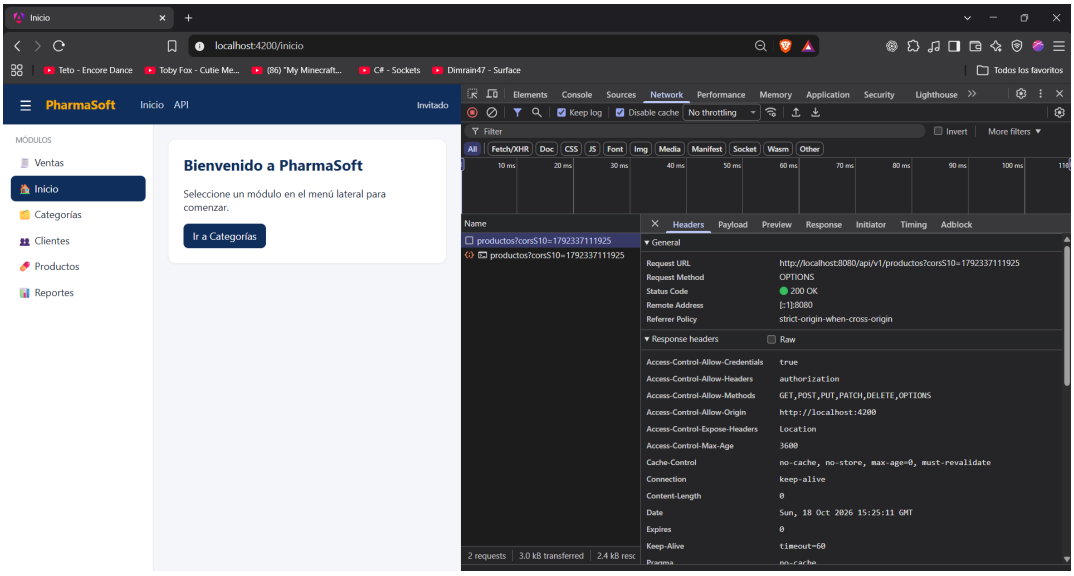


Figura 3. C-01: preflight OPTIONS 200 desde localhost:4200 con Authorization autorizado.

Referencias de implementación: AuthController, AuthServiceImpl.cambiarPassword, UsuarioServiceImpl.asignarRoles/cambiarEstado, JwtAuthenticationFilter.doFilterInternal, JwtService, SecurityConfig y ReporteController. Pruebas: SeguridadIntegrationTest, JwtServiceTest y PermisosPorRolTest. Matriz de 28 casos y evidencia por aspecto en el documento complementario.